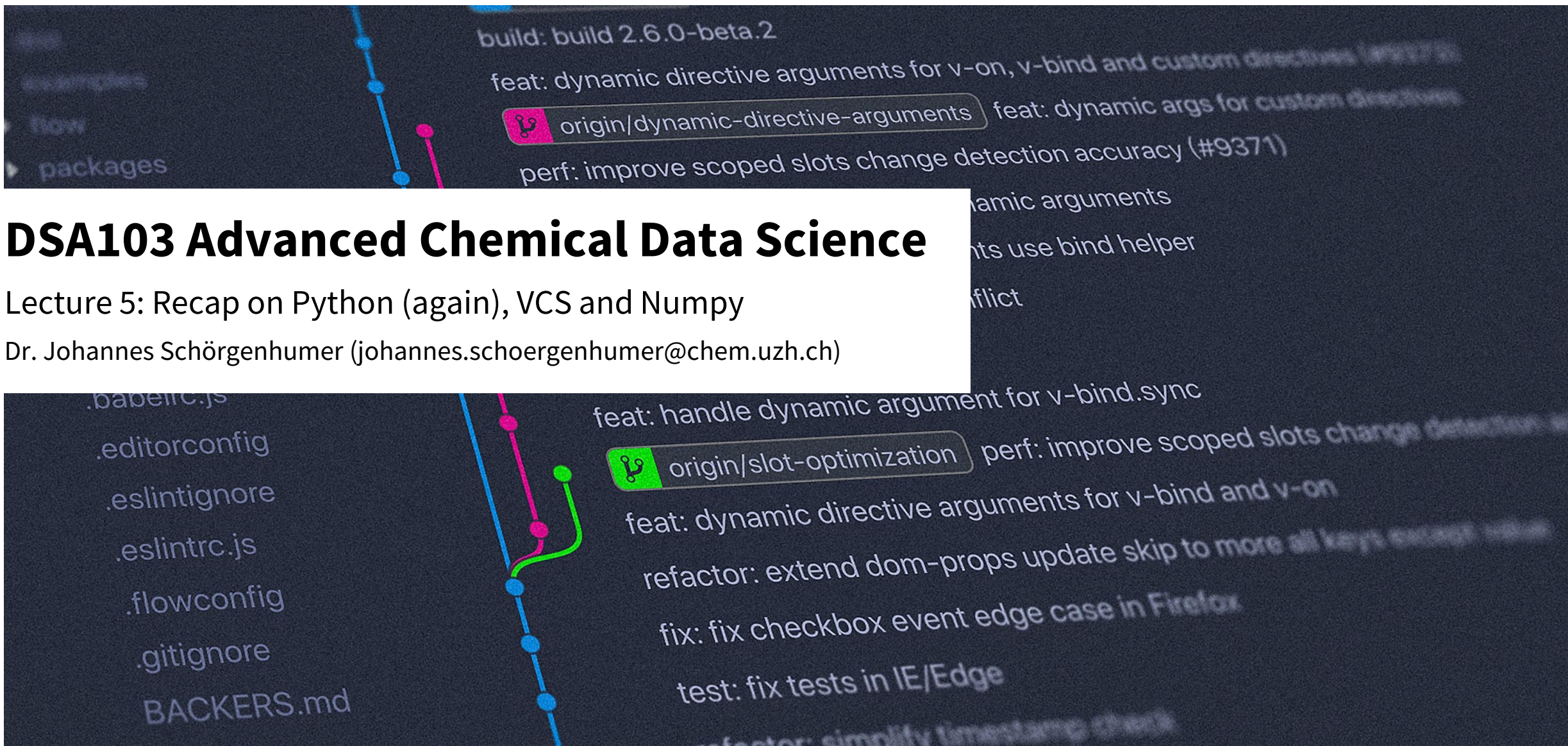




DSA103 Advanced Chemical Data Science

Lecture 5: Recap on Python (again), VCS and Numpy

Dr. Johannes Schörgenhumer (johannes.schoergenhummer@chem.uzh.ch)



Content overview

Python review continued:

- Objects and types
- Binary numbers and float precision
- Classes

VCS in review and application

Numpy recap

Objects and types again

Everything in Python (except keywords) is an object with a type:

```
# Scalar types (single values)
```

```
concentration = 0.5      # float
num_atoms = 6           # int
element = "H"           # string
is_stable = True        # boolean
```

```
# Non-scalar types (collections)
```

```
concentrations = [0.1, 0.2, 0.3]      # list
compound_data = {"name": "water", "formula": "H2O"} # dictionary
```

```
# Check types
```

```
print(type(concentration)) # <class 'float'>
print(type(concentrations)) # <class 'list'>
```

```
# Type comparison
```

```
print(isinstance(concentration, float)) # True
print(isinstance(concentration, list)) # False
```

```
# Membership operators
```

```
"H" in ["H", "O", "N"] # True
```

```
"formula" in {"name": "water", "formula": "H2O"} # True
```

What type is this?

```
# Each experiment is a dictionary
```

```
experimental_data = [
    {"temperature": 25.0, "catalyst": 0.1, "yield": 45.2, "cost": 10.5},
    {"temperature": 50.0, "catalyst": 0.1, "yield": 67.8, "cost": 12.0},
    {"temperature": 75.0, "catalyst": 0.1, "yield": 82.3, "cost": 15.5},
    {"temperature": 100.0, "catalyst": 0.1, "yield": 78.9, "cost": 20.0},
    {"temperature": 50.0, "catalyst": 0.2, "yield": 75.4, "cost": 18.0},
    {"temperature": 75.0, "catalyst": 0.2, "yield": 89.1, "cost": 22.5}
]
```

What would be an alternative to this data structure?

NumPy Array, Pandas DataFrame, Python Class

Binary numbers and float precision

```
from typing import Dict

def calculate_mass_from_dict(formula_dict: Dict[str, int], masses: Dict[str, float]):
    """Calculate molecular mass from dictionary format"""
    return sum(masses[atom]*amount for atom, amount in formula_dict.items())

calculate_mass_from_dict({"H": 2, "O": 1}, atomic_masses)

# Returns 18.015 here on Windows 11, X64
# 18.016 below is probably a typo but it could also be fun with floats :)

# Example: {"H": 2, "O": 1} should return 18.016
```

Computers store numbers in binary (base 2), which can cause precision issues with decimal numbers:

Float precision issues

```
print(0.1 + 0.2) # Output: 0.30000000000000004 (not exactly 0.3!)
```

```
print(0.1 + 0.2 == 0.3) # Output: False
```

Better way to compare floats

```
def close_enough(a, b, tolerance=1e-9):
    return abs(a - b) < tolerance
```

```
print(close_enough(0.1 + 0.2, 0.3)) # Output: True
```

Comparison operators that handle precision

```
import math
math.isclose(0.1 + 0.2, 0.3) # Better than == for floats
```

```
math.isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)
```

Return `True` if the values `a` and `b` are close to each other and `False` otherwise.

<https://docs.python.org/3/library/math.html>

Keywords not covered last time: Class, raise

Class = user-defined template for creating objects (instances); make code more structured, flexible, user-friendly, maintainable, ...

Classes contain attributes and methods, which can be inherited by Child classes:

- **Attributes:** data of class or instances, accessed by **obj.attribute_name**
- **Methods:** functions typically working on class data, accessed by **obj.method_name()**

Important methods:

- **__init__()**: constructor, automatically executed when object is created, initialises attributes
- **__str__()**: define a custom, verbose informal string representation of an object

ClassNameConvention: Capitalise every word, no underscore (unlike methods and functions)

When to avoid Classes:

- If only used for data storage
- If you have only one method

More comprehensive guide to classes incl. examples:
<https://realpython.com/python-classes/>

```
class Atom:
    def __init__(self, species: str):
        self.species = species

    def mass(self) -> float:
        # average atomic weights from
        Wikipedia
        if self.species == "H":
            return 1.008
        elif self.species == "C":
            return 12.011
        elif self.species == "O":
            return 15.999
        else:
            raise ValueError(f"Unknown
species: {self.species}")

hydrogen = Atom("H")
```

Activity: Float precision and Classes

1) Float precision...

```
from typing import Dict

def calculate_mass_from_dict(formula_dict: Dict[str, int], masses: Dict[str, float]):
    """Calculate molecular mass from dictionary format"""
    return sum(masses[atom]*amount for atom, amount in formula_dict.items())

calculate_mass_from_dict({"H": 2, "O": 1}, atomic_masses)
# Returns 18.015 here on Windows 11, X64
# 18.016 below is probably a typo but it could also be fun with floats :)
# Example: {"H": 2, "O": 1} should return 18.016
```

2) Classes

Recap VCS – Motivation: Manual Version Control

Imagine working on your thesis...

- Thesis.docx

When you have most of the text in, you try some formats:

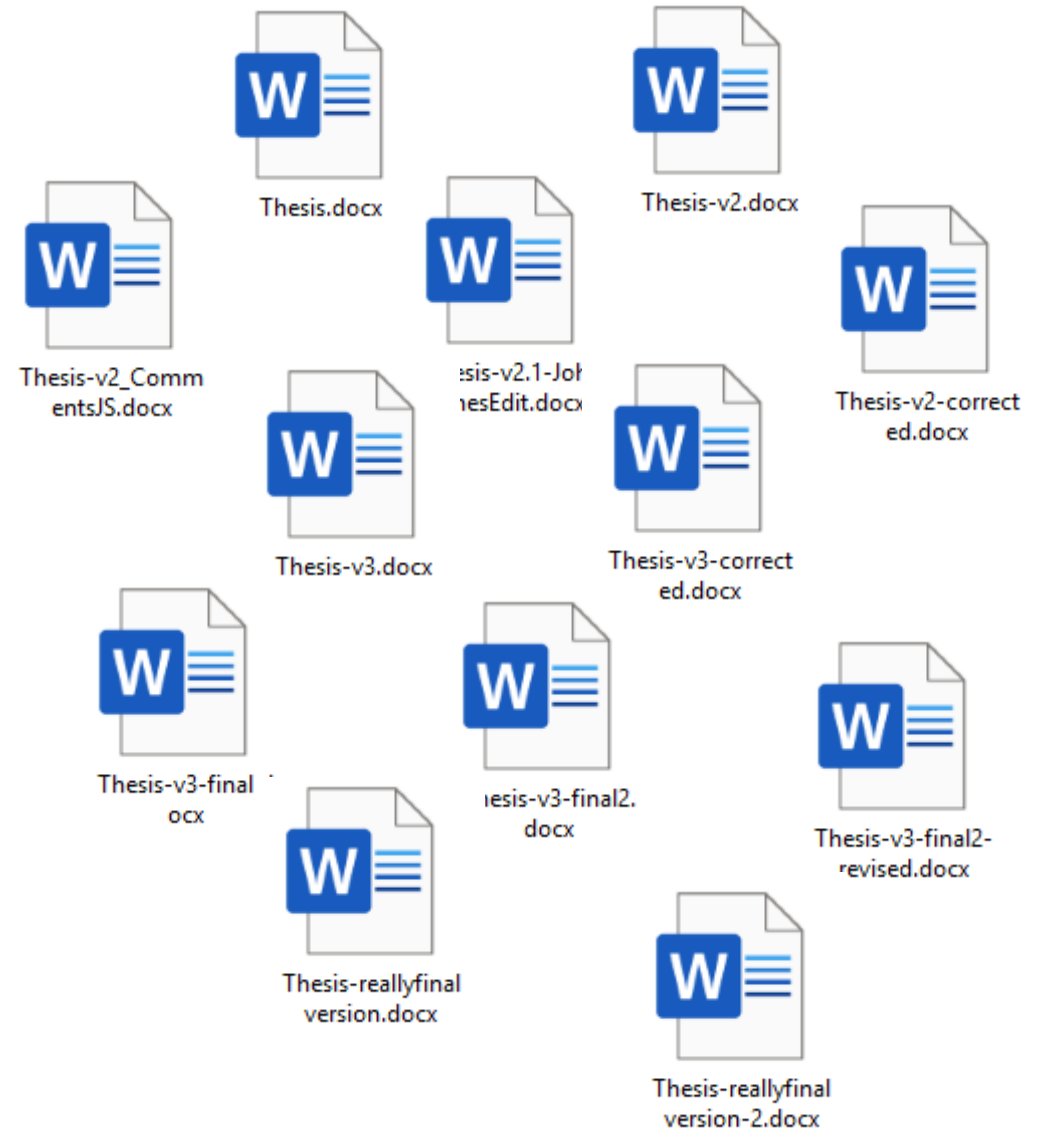
- Thesis-v2.docx

Then you send it to some colleagues for proof reading:

- Thesis-v2_CommentsJS.docx
- Thesis-v2.1-JohannesEdit.docx
- Thesis-v2-corrected.docx

You consolidate all comments:

- Thesis-v3.docx
- ...



Version Control Systems (VCS)

Benefits of VCS:

- Tracking of changes without keeping multiple files
- Long-term time change history: Hopping between different timepoints in development
- Branching out for testing, merging for integration and revert changes if unsuccessful
- Traceability: Changes and contributors

But there is more in the bigger picture:

- Co-development efficiency
- Risk management (safety net)
- Transparency
- Improve quality
- Enable integration of code / data
- Enable automation of tasks

What are Version Control Systems (VCS)

VCS are indispensable tools for coding and the backbone of collaborative projects in the field of programming and data science

- Concept was introduced early on: precursors in 60s, IBM's Source Code Control System in 1972
- Landscape was revolutionized in 2005 with the introduction of git by Linus Torvalds
- Many different tools emerged to deal with specific development requirements (git is not the only example)
- Examples: Git, Subversion(SVN), Mercurial, Azure DevOps Server, ...



Linus Torvalds (2002) - Linuxmag.com, Dezember 2002, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=17991>

What are Version Control Systems (VCS)

Types:

- Centralized VCS (e.g. SVN, kind of built-in in MS Office 365):
 - All versions are saved on a centralized server
 - All changes are committed and pushed directly to the server
 - Ease of management, but no offline coding possible
- Distributed VCS (e.g. Git):
 - Every user has a complete copy of the repository
 - Working offline is possible
 - Robust framework for merging and branching
 - Improves efficiency and redundancy



Git vs. GitLab vs. GitHub

Git: Opensource VCS, basis for other tools:

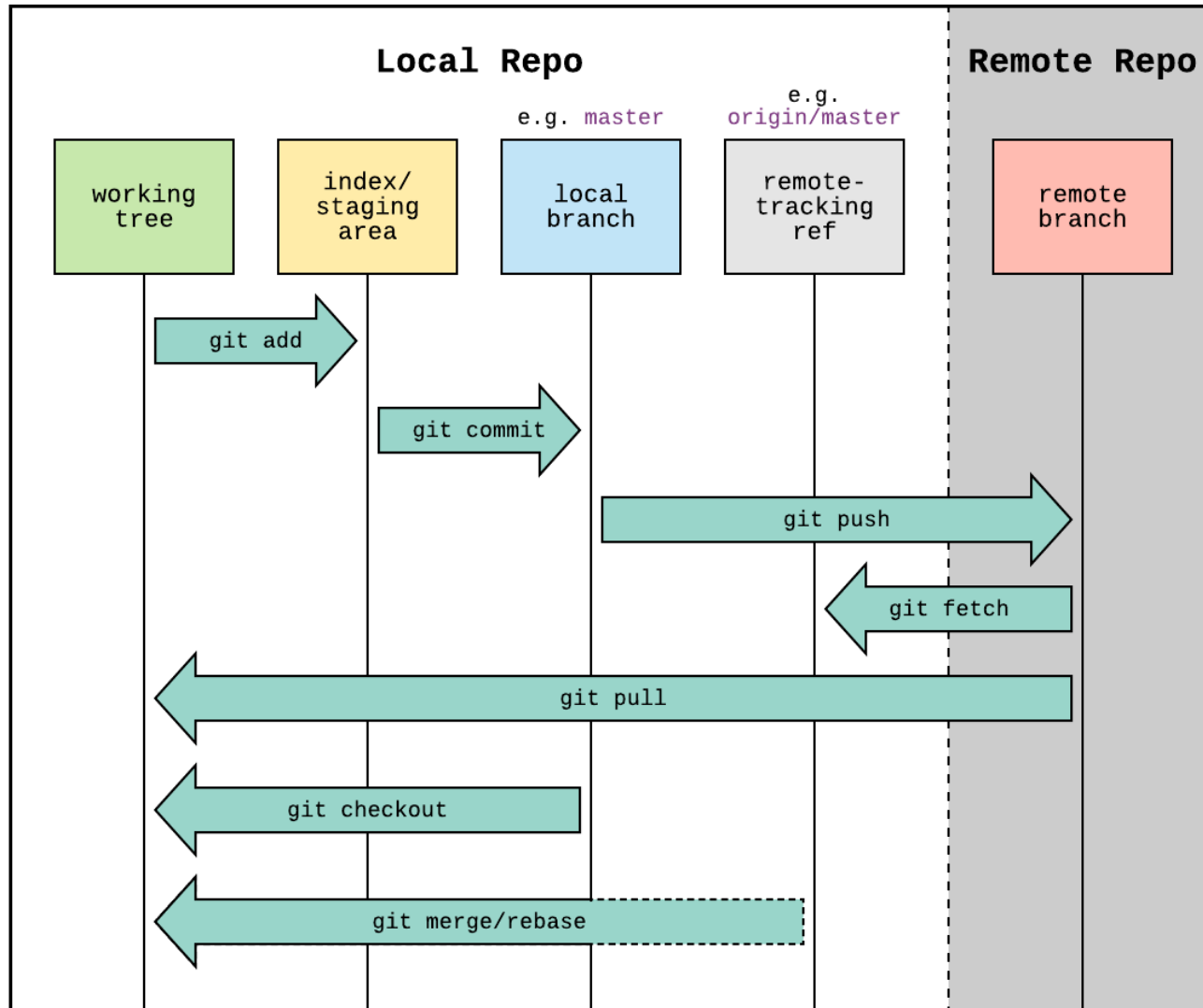
GitLab: web-based platform to streamline development workflows. Includes git repository management and combines with continuous integration/continuous development (CI/CD) to automate entire delivery process

GitHub: Cloud-based hosting service for managing git repositories, focused on sharing with collaborators or making codebase available publicly (Social coding)

We will be using GitHub! (arguably most common platform for sharing data in scientific community) for this you need to understand the basics of git...



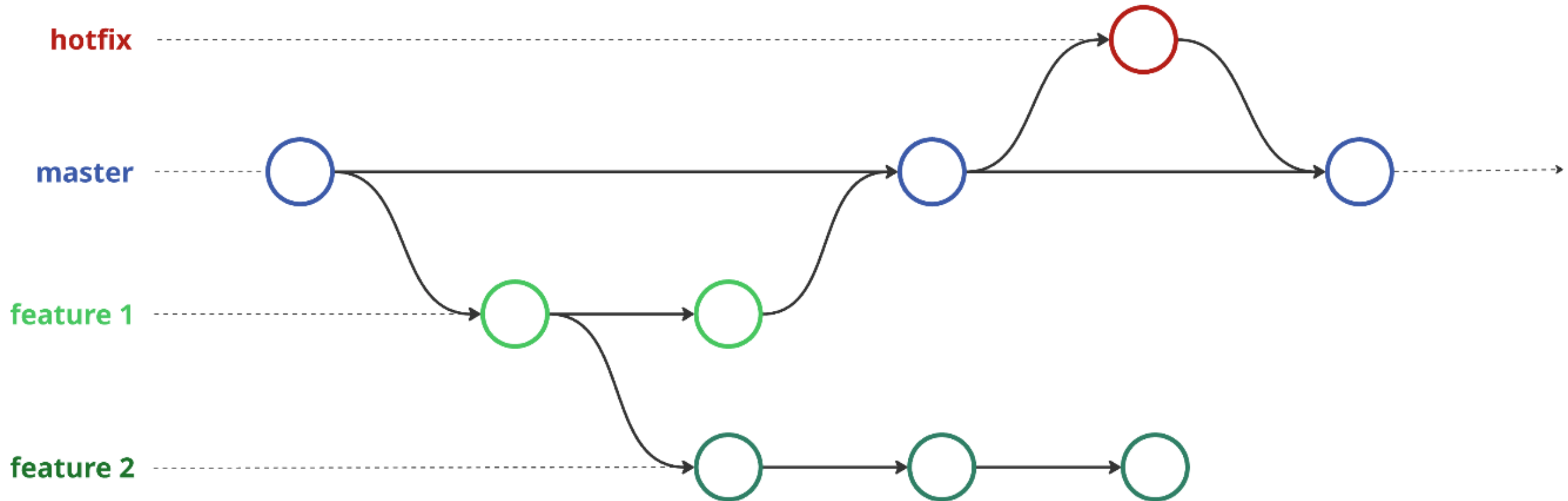
How Git works - Git basics



Cheat sheet: <https://education.github.com/git-cheat-sheet-education.pdf>

- Repository/"Repo": digital storage for project files and complete change history (tracked commits)
- Commit: Snapshot of project at specific point in time, any changes are tracked
- Branch: Parallel version of your project to work on different features or fixes independently
- Checkout: Switching between branches or specific commits in your repository
- Merge: Combining changes from one branch into another, usually to integrate new features or fixes
- Conflict: A situation where Git cannot automatically merge changes due to overlapping edits
- Fork: personal copy of someone else's repo, changes do not affect the original repo (without pull request)
- Clone: Creates a local copy of a remote repository
- Pull: Fetching and integrating changes from a remote repository into your local repository
- Push: Sending your committed changes from your local repository to a remote repository

Branches and Merging

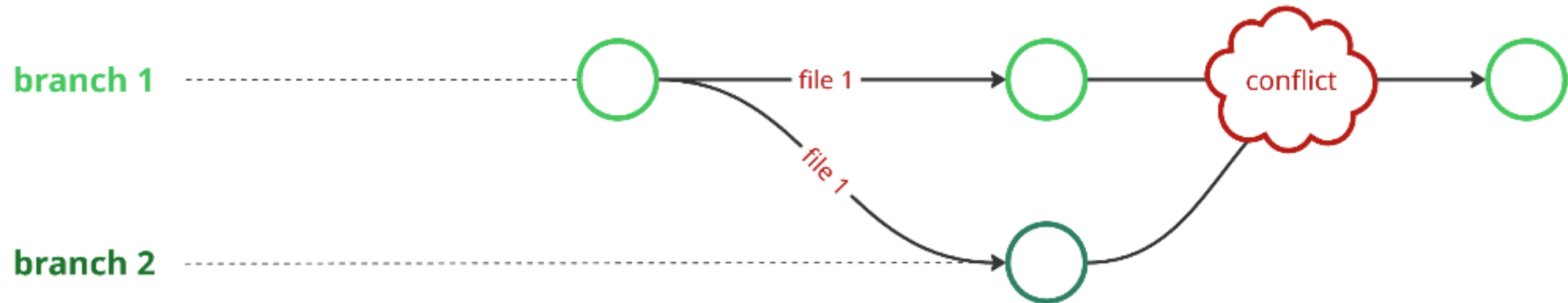


Branches and Merging

Merge conflicts arise if the same file is changed at the same time.

Need to be resolved manually.

Best to avoid by clearly structured branches.



Activity: GitHub & VCS in VSC 😊

Create a repository via Github, clone and set up uv project (uv init)

Create a python file in the main branch (e.g. use the code on the upper right), then commit

Create a branch in your local repository, create a second file (e.g. a runscript as indicated on the lower right), commit.

Merge the development branch into the master branch (main)

Create a merge conflict: modify the same file and commit on two different branches

Resolve merge conflicts

Push to remote repo

```
def greetings(name: str) → str:
    """Greetings by python"""
    return f"Hello {name}!"

if __name__ == "__main__":
    name = input("What is your name? ")
    print(greetings(name))
```

```
from greetings_script import greetings

print(greetings("World"))
```

Best practices

The repository setup: Define environment, access control (Templates)

Establish workflow alignment: Define coworking rules

Write clear commit messages: Explain the changes in short and concisely. Use standardized nomenclature (e.g. conventionalcommits)

Commit often: Small commits, concise messages.

Test before committing: Use linters (code formatting!) and testing tools (e.g. pytest) and CI workflows to do that automatically

Commit only when ready

Avoid conflicts: Always use up-to-date code, make use of branches!

Use branches: Code branches support multiple versions of software releases and patches. Avoid committing directly to main!

Limit repository access: Only to contributors who need it, control unauthorized access

<https://github.com/resources/articles/software-development/what-is-version-control>

Best practices – useful tools

Project templates (cookiecutters): Provide standardised and consistent environment for working on a project, many different templates available, e.g. for python scripts, Django or data science (<https://www.cookiecutter.io/>)

Package managers:

- Help to **define, lock, install, and reproduce** project dependencies reliably and efficiently. Examples: pip, uv, conda
- Key aspects: **dependency management** (auto-install needed packages), **lockfiles** (exact versions of dependencies ensure reproducibility), **virtual environments** (isolation from system environment), **installability** (lets others install your package with one command, e.g. `uv pip install mypackage`)

Installable repos / packages: follow clear data structure (*vide infra*), include .toml file to declare dependencies and a lockfile to specify versions. Voilà!

```
my-project/
├── my_package/          # Your Python code (must have __init__.py)
│   └── __init__.py
├── pyproject.toml       # Project metadata + dependencies
├── README.md           # Project description
├── .gitignore
└── uv.lock              # (Optional) Lock file for reproducibility
```

Linters: Analyse code for style errors, bad practices and style violations (e.g. flake8, ruff)

Formatters: Rewrite / modify code to follow consistent style such as line breaks, etc. (e.g. ruff)

NumPy

Numpy = library for supporting large multidimensional arrays and matrices (including functions to operate on these arrays)

Main benefit:

- Vanilla lists: heterogeneous – each item must contain its own type info additional to the data
- Numpy arrays: homogeneous, each item of the same type – much more efficient for storing and manipulating big data

Usual alias is **np**

```
import numpy as np  
np.__version__
```

✓ 0.0s

'2.0.2'

NumPy – creating arrays

Create arrays: `np.array`

- One-dimensional arrays
 - only one index is needed to access entries, e.g. **`onedim[0]`**
- Two-dimensional arrays:
 - 2D arrays have *rows* (horizontal) and *columns* (vertical)
 - two indices are needed: **`array[row, column]`**

dtype specifies the data type in the array!

List of dtypes: <https://numpy.org/doc/stable/user/basics.types.html>

```
onedim = np.array([1, 2, 3, 4, 5])
print(onedim)
```

✓ 0.0s

```
[1 2 3 4 5]
```

```
twodim = np.array([[1, 2, 3], [4, 5, 6]])
print(twodim)
```

✓ 0.0s

```
[[1 2 3]
 [4 5 6]]
```

```
onedim = np.array([1, 2, 3, 4, 5], dtype='float32')
print(onedim)
```

✓ 0.0s

```
[1. 2. 3. 4. 5.]
```

NumPy – creating arrays

Other options to create arrays:

- **`np.arange(start, end, stepsize)`**
- **`np.zeros((rows, columns))`**
- **`np.ones((rows, columns))`**
- **`np.random.random((rows, columns))`**
- **`np.random.randint(low, high, (rows, columns))`**

NumPy – array attributes

The type of an array is `numpy.ndarray`

```
onedim = np.array([1, 2, 3, 4, 5])  
type(onedim)
```

✓ 0.0s

`numpy.ndarray`

Useful attributes to inspect:

- **`np.ndim`**(array): number of dimensions
- **`np.shape`**(array): size of each dimension
- **`np.size`** (array): total size, i.e. number of all elements

NumPy – 1D slicing

The NumPy slicing syntax follows that of the standard Python list: **x[start:stop:step]**. Note: The stop index will not be included!

Slice	Description
x[:5]	First five elements
x[5:]	All elements after index 5
x[4:7]	Middle subarray
x[::2]	Every other element
x[1::2]	Every other element, starting at index 1
x[::-1]	All elements, reversed
x[5::-1]	Reverses all elements up until index 5 (included)

Slicing can be used to change one or multiple elements (also in 2D arrays):

```
# set the last three values to 333
nums = np.array([1,2,3,4,5, 20, 100, 23, 43, 12])
nums[:-4:-1] = 333
print(nums)
```

✓ 0.0s

```
[ 1  2  3  4  5 20 100 333 333 333]
```

NumPy – 2D slicing

Multiple slices are now separated by commas, first stating the slices of **rows**, then of the **columns**: **Y[start:stop:step, start:stop:step]**

Slice	Description
Y[:2, :3]	First two rows and first three columns
Y[:3, ::2]	First three rows and every other column
Y[::-1, ::-1]	Reverse rows and columns
Y[:, 1]	Second column
Y[3, :]	Fourth row
Y[3]	Same as Y[3, :] , fourth row

NumPy – manipulation of arrays

Copy vs. View!

- With Python **lists**, the slices will be ***copies (duplicated data)***: If we modify the subarray (e.g. change one entry), only the copy gets changed
- With NumPy **arrays**, the slices will be ***direct views (original data)***: If we modify the subarray (view), the original array gets changed too
- When working with large datasets, we don't need to copy any data (slow operation)
- Creating copies: we can use the **`copy()`** method of a slice to create a copy of the specific subarray
- *Note*: The type of a slice is again **`numpy.ndarray`**

NumPy - ufuncs

Looping over normal lists in vanilla python is very slow due to the type checking for each element

Numpy **universal functions (ufuncs)** are faster, vectorized operation for the whole array instead of a classical loop.

Operator	Equivalent ufunc	Description
+	np.add	Addition
-	np.subtract	Subtraction
-	np.negative	Unary negation (e.g., -2)
*	np.multiply	Multiplication
/	np.divide	Division
//	np.floor_divide	Floor division (e.g., 3 // 2 = 1)
**	np.power	Exponentiation (e.g., 3**2 = 8)
%	np.mod	Modulus/remainder (e.g., 9 % 4 = 1)

NumPy – aggregates

Aggregates are useful to compute summary statistics for the data, such as:

- Mean
- Standard deviation
- Median
- Minimum
- Quantiles
- ...

Universal function: compare vanilla operations with vectorized ufuncs (%timeit)

UZH ZI & David Pinezich, ZI APPD course 2024

Function name	Description
np.sum	Calculates sum of elements
np.prod	Calculates product of elements
np.mean	Computes mean of elements
np.std	Calculates standard deviation
np.min	Yields minimum value in elements
np.max	Yields maximum value in elements
np.argmin	Finds index of minimum value
np.argmax	Finds index of maximum value
np.median	Computes median of elements

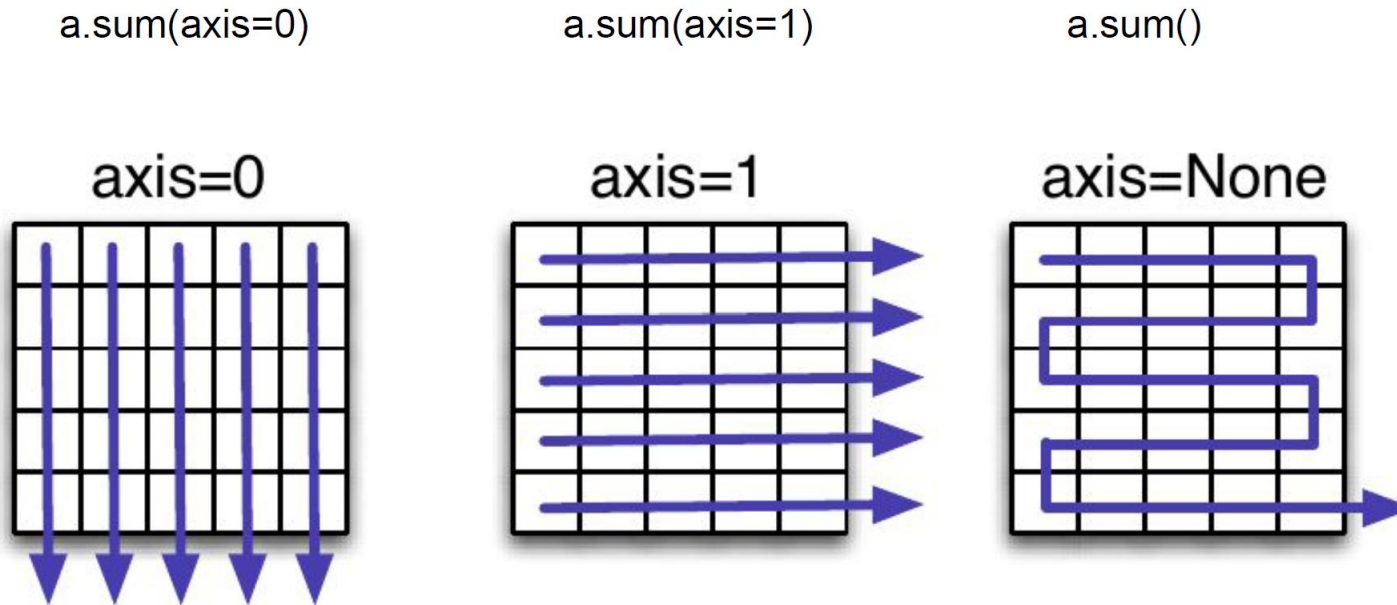
```
%timeit np.sum(random_array) # Numpy ufunc
%timeit sum(random_array) # Python function
✓ 18.4s

1.84 µs ± 19.9 ns per loop (mean ± std. dev. of 7 runs, 1,000,000 loops each)
4.41 µs ± 135 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
```

NumPy – multidimensional aggregates

By default, aggregation functions will be applied to the whole array

For multidimensional arrays, an argument can be provided specifying the axis for the operation:



NumPy – Boolean arrays

NumPy implements comparison operators as element-wise ufuncs, which always results in an **array with a Boolean data type**:

```
np.array([1, 2, 3]) < 2
✓ 0.0s
array([ True, False, False])
```

It is possible to do an element-to-element comparison between two arrays:

False will be interpreted as 0, therefore, the **True** elements can be counted by either the **count_nonzero()** or **sum()** function.

np.all() and **np.any()** will return the Boolean if the condition is met by all or any element, respectively.

Operator	Equivalent ufunc
<code>==</code>	<code>np.equal</code>
<code>!=</code>	<code>np.not_equal</code>
<code><</code>	<code>np.less</code>
<code><=</code>	<code>np.less_equal</code>
<code>></code>	<code>np.greater</code>
<code>>=</code>	<code>np.greater_equal</code>

```
np.array([3,0,8,6,2]) > np.array([2,0,6,6,1])
✓ 0.0s
array([ True, False,  True, False,  True])
```

```
nums = np.array([1,2,3,4,5])
np.count_nonzero(nums < 4)
✓ 0.0s
3
```

NumPy – Boolean arrays

NumPy also implements bitwise logic operators as element-wise ufuncs, which can be used to construct compound conditions (consisting of multiple conditions):

Operator	Equivalent ufunc
&	<code>np.bitwise_and</code>
	<code>np.bitwise_or</code>
^	<code>np.bitwise_xor</code>
~	<code>np.bitwise_not</code>

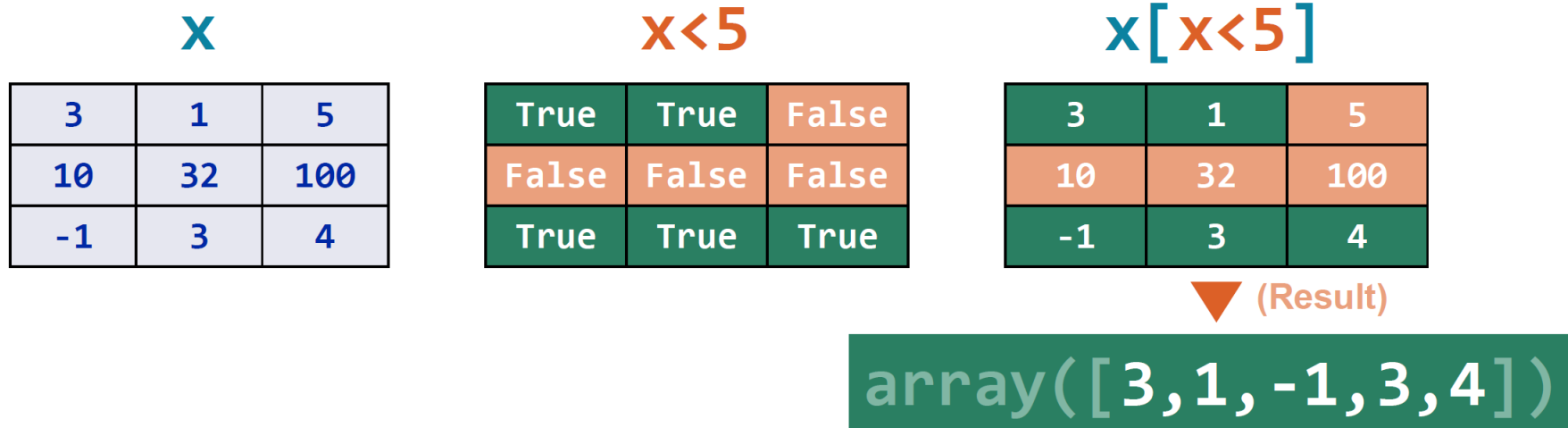
```
nums = np.array([1,2,3,4,5])
# set all values to 111 for which both conditions are true
nums[(nums < 3) & (nums < 2)] = 111
print(nums)
```

✓ 0.0s

[111 2 3 4 5]

NumPy – Boolean masks

Boolean arrays from a comparison can be used as **mask to select entries** that meet the same condition:



Exercises: Numpy and collaborative coding

Work in pairs:

- 1) Use one of the Github repos you created for the VCS part, copy the two Numpy files (numpy_exercise_A, numpy_exercise_B) from the DSA folder into the repo
- 2) Invite your partner as collaborator to the repo on Github
- 3) You and your partner work on the two notebooks A and B independently, but in different branches.
- 4) Merge branches and compare the results of the exercises